МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"
Кафедра «Систем автоматизованого проектування»



ЗВІТ

до лабораторної роботи №10

з дисципліни "Технології веб-розробки та дизайну"

на тему:

«Інтеграція динамічних медіакомпонентів та JS-бібліотек»

Освітній вебпроєкт: SimpleArduino

Тематичний варіант: «Онлайн-освіта / Інтерактивна платформа курсів з робототехніки та Arduino»

Публічний URL сайту (Vercel): https://simplearduino-lab8.vercel.app

Публічний URL сайту (GitHub Pages): https://nazarbuliga.github.io/simplearduino-lab8/

Репозиторій коду (GitHub): https://github.com/nazarbuliga/simplearduino-lab8

Студент:
групи ПП-25
Булига Назарій

Оцінка: _____

Підпис викладача: _____

Керівник:
доцент кафедри САПР
Паращин Ж.Д.

«___» _____ 2026 р.

Львів — 2026

1. МЕТА ТА ЗАВДАННЯ ЛАБОРАТОРНОЇ РОБОТИ

Мета роботи: Опанувати практичні навички пошуку, підбору, підключення через CDN (Content Delivery Network), налаштування за допомогою об'єктів конфігурації та глибокої CSS-кастомізації популярних JavaScript-бібліотек і динамічних медіакомпонентів. Навчитись інтегрувати сучасні сенсорні слайдери (Swiper.js), модальні лайтбокси/галереї (Fancybox v5) та інтерактивні текстові анімації (Typed.js) у готову кодову базу освітнього вебсайту SimpleArduino, забезпечуючи високу швидкість завантаження, плавність рендерингу без стрибків макета (Zero Cumulative Layout Shift / FOUC), адаптивність та суворе дотримання стандартів веб-доступності (WCAG 2.1 AA).

Завдання лабораторної роботи:
1. Етап 1 (CDN-інфраструктура): Підключити мініфіковані CSS-стилі плагінів у секції <head> та JS-файли перед закриваючим тегом </body> через надійну розподілену мережу CDN jsDelivr;
2. Етап 2 (Інтерактивний адаптивний слайдер): Реалізувати на базі Swiper.js v11 адаптивну карусель відгуків спільноти мейкерів SimpleArduino на 6 карток із нескінченним циклом (loop), автоматичним гортанням (autoplay), зупинкою при наведенні курсору (pauseOnMouseEnter), клікабельною динамічною пагінацією буллетами та кнопками навігації;
3. Етап 3 (Модальна медіагалерея схемотехніки): Інтегрувати сучасний лайтбокс Fancybox v5.0 для 6 технічних схем апаратних проєктів Arduino зі 100% відповідністю графіки до технічних описів, підтримуючи повнорозмірний перегляд (Zoom), повноекранний режим (Fullscreen), панель мініатюр (Thumbs) та україномовну локалізацію (l10n);
4. Завдання підвищеної складності (Друкований ефект): Впровадити в Hero-секцію бібліотеку Typed.js v2.1 для живої друкованої анімації коду мікроконтролера C++ із миготливим термінальним курсором у стабілізованому контейнері фіксованої висоти;
5. Оптимізація плавності завантаження (Zero Layout Shift): Усунути дьоргання при первинному завантаженні шляхом згладжування Hero-анімацій, попередньої стилізації слайдера (Pre-init Flexbox) та жорсткої фіксації пропорцій зображень (aspect-[16/10] з object-contain);
6. Архітектурна ізоляція та модульність: Організувати ініціалізацію в окремому файлі js/plugins-init.js з використанням події DOMContentLoaded та захисних перевірок наявності елементів у DOM;
7. Стилізація та кастомізація: Перевизначити дефолтні стилі бібліотек у файлі css/custom-styles.css відповідно до палітри кольорів та UI-кита освітньої

платформи;
8. Інструментальне тестування: Провести всебічний аудит консолі Google Chrome DevTools (0 errors) та автоматизоване тестування за допомогою headless-браузера Puppeteer на мобільних і десктопних в'юпортах.

2. КОРОТКИЙ ОПИС ВЕБСАЙТУ ТА АРХІТЕКТУРИ ПРОЄКТУ

Освітній портал SimpleArduino (https://simplearduino-lab8.vercel.app) є інтерактивною платформою для навчання мікроконтролерів Arduino, схемотехніки, IoT та операційної системи реального часу FreeRTOS. Проєкт розроблено згідно з філософією сучасної веб-розробки: семантична розмітка HTML5, дизайн-система на базі Tailwind CSS та інтерактивні модулі на чистому JavaScript.

У Лабораторній роботі №10 функціонал платформи було якісно розширено додаванням динамічних інтерактивних медіакомпонентів:

Компонент / Бібліотека	Місце в інтерфейсі	Функціональне призначення
Swiper.js v11 (Modern Touch Slider)	Секція «Спільнота SimpleArduino» (Відгуки користувачів)	Адаптивна сенсорна карусель 6 відгуків з автоплеєм (3.8 с), зупинкою при ховері, динамічними буллетами та брейкпоінтами (1, 2, 3 слайди).
Fancybox v5.0 (Lightbox & Media Gallery)	Секція «Галерея схем та макетів» (#gallery)	Модальний перегляд високої чіткості для 6 інженерних фотографій стендів і схем з зумом коліщатком миші, панеллю мініатюр та українською локалізацією.
Typed.js v2.1 (Typewriter Engine)	Головна Hero-секція сайту (Термінальний рядок C++)	Інтерактивний друк фрагментів скетчів мікроконтролера в реальному часі зі швидкістю 40 мс/символ та циклічним повторенням у контейнері з фіксованою висотою.

3. ХІД ВИКОНАННЯ РОБОТИ ТА ПРОГРАМНА РЕАЛІЗАЦІЯ

3.1. Підключення CDN-ресурсів та оптимізація рендерингу (FOUC)

Для забезпечення високої швидкості завантаження сайту, зменшення обсягу репозиторію та використання глобального кешу браузера підключення бібліотек виконано через мережу jsDelivr. Для уникнення візуального ефекту Flash of Unstyled Content (FOUC) файли таблиць стилів CSS підключено в середині тегу <head>:

```
<!-- Сторонні бібліотеки: Swiper v11 та Fancybox v5 (Лабораторна робота №10) -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/swiper@11/swiper-
bundle.min.css" />
<link rel="stylesheet"
href="https://cdn.jsdelivr.net/npm/@fancyapps/ui@5.0/dist/fancybox/fancybox.css" />
```

JavaScript-файли бібліотек підключено перед закриваючим тегом </body> у суворій відповідності до порядку залежностей:

```
<!-- CDN Скрипти сторонніх JS-бібліотек (Лабораторна робота №10) -->
<script src="https://cdn.jsdelivr.net/npm/swiper@11/swiper-bundle.min.js"></script>
<script
src="https://cdn.jsdelivr.net/npm/@fancyapps/ui@5.0/dist/fancybox/fancybox.umd.js"></scri
pt>
<script src="https://cdn.jsdelivr.net/npm/typed.js@2.1.0/dist/typed.umd.js"></script>
<script src="js/plugins-init.js"></script>
<script src="js/validation.js"></script>
```

3.2. Усунення дьоргання при завантаженні (Zero Layout Shift / Smooth UX)

Для забезпечення плавного, приємного та стабільного завантаження сторінки без ривків контенту (усунення дефекту Cumulative Layout Shift) впроваджено комплекс інженерних оптимізацій:
1. Згладжування Hero-анімацій: агресивне вертикальне зміщення translateY(22px) замінено на делікатний fade-in з мікро-зсувом translateY(6px), що ліквідувало стрибки заголовків;
2. Стабілізація висоти Typed.js: контейнеру #typed-code-runner призначено фіксовану висоту h-8 min-h-[32px] та властивість overflow-hidden whitespace-nowrap, що запобігає зсувам блоків при зміні рядків коду;
3. Пре-стилізація Swiper.js: ще до ініціалізації JavaScript контейнер .reviews-swiper має flex-розкладку карток з точним розрахунком ширини (100% mobile, 50% tablet, 33.3% desktop), завдяки чому слайдер не згортається і не

стрибає під час завантаження скрипта;
4. Фіксація геометрії зображень: для карток схемотехніки задано жорсткі пропорції aspect-[16/10] з фоном bg-slate-900 та властивістю object-contain, що усуває будь-яке обрізання деталей схем.

3.3. Реалізація адаптивного сенсорного слайдера Swiper.js v11

У блоці відгуків мейкерів було реалізовано адаптивний слайдер. Контейнер відгуків розміщено у спеціальній обгортці з кастомними навігаційними стрілками та динамічними буллетами. Для кожного слайда створено семантичну картку <figure> з цитатою, рейтингом зірочками, аватаром автора та підписом.

```
// Об'єкт конфігурації Swiper.js v11 в js/plugins-init.js
const swiperContainer = document.querySelector('.reviews-swiper');
if (swiperContainer && typeof Swiper !== 'undefined') {
  new Swiper('.reviews-swiper', {
    loop: true,                  // Нескінченне циклічне гортання
    speed: 650,                  // Плавність анімації (650 мс)
    grabCursor: true,            // Курсор-рука для десктопів
    watchOverflow: true,         // Блокування кнопок при нестачі слайдів
    spaceBetween: 20,            // Відступ між картками у пікселях

    autoplay: {
      delay: 3800,               // Інтервал автогортання 3.8 секунди
      disableOnInteraction: false, // Відновлення автоплею після свайпу
      pauseOnMouseEnter: true,   // Зупинка автоплею при наведенні курсору
    },

    pagination: {
      el: '.swiper-pagination',
      clickable: true,           // Перехід до слайда за кліком на буллет
      dynamicBullets: true,      // Динамічне масштабування булетів
    },

    navigation: {
      nextEl: '.swiper-button-next',
      prevEl: '.swiper-button-prev',
    },

    breakpoints: {
      320: { slidesPerView: 1, spaceBetween: 16 }, // Мобільні пристрої
      640: { slidesPerView: 2, spaceBetween: 20 }, // Планшети
      1024: { slidesPerView: 3, spaceBetween: 24 } // Десктопи
    },

    keyboard: { enabled: true, onlyInViewport: true },
    observer: true,
    observeParents: true,
  });
}
```

3.4. Реалізація модальної медіагалереї схемотехніки Fancybox v5.0

Секція #gallery містить 6 інтерактивних карток, де кожне зображення строго відповідає заявленому проєкту:

№	Бейдж	Заголовок схеми	Файл та автентичний вміст
1	#01 Реал-тест	Arduino Uno: Апаратний стенд	arduino-real-hardware.jpg (реальне фото плати зі світлодіодом на макетці)
2	#02 Кнопка	Тактова кнопка & Деренчання	tinkercad-button-circuit.png (кнопка з підтягувальним резистором 10 кОм)
3	#03 Світлофор	Світлофор: Модель Tinkercad	tinkercad-traffic-light.png (3 світлодіоди Red, Yellow, Green та кнопка)
4	#04 Потенціометр	Потенціометр & Дільник напруги	tinkercad-potentiometer.png (змінний резистор, пін A0 Arduino)
5	#05 Сонар	Сонар HC-SR04: Відлуння	tinkercad-ultrasonic.png (ультразвуковий датчик 40 кГц, Trigger та Echo)
6	#06 I2C Екран	Символьний екран LCD 1602 I2C	tinkercad-i2c-lcd.png (символьний дисплей на 2-провідній шині I2C)

Конфігурація Fancybox v5.0 з локалізацією:

```
// Конфігурація Fancybox v5.0 з локалізацією в js/plugins-init.js
if (typeof Fancybox !== 'undefined') {
  Fancybox.bind('[data-fancybox="hardware-gallery"]', {
    animated: true,
    showClass: 'f-fadeIn',
    hideClass: 'f-fadeOut',

    Toolbar: {
      display: {
        left: ['infobar'],
        middle: ['zoomIn', 'zoomOut', 'toggle1to1', 'rotateCCW', 'rotateCW', 'flipX',
'flipY'],
        right: ['slideshow', 'thumbs', 'fullscreen', 'close'],
      },
    },

    Thumbs: {
      autoStart: true,           // Автопоказ панелі мініатюр внизу екрана
      type: 'modern',
```

```
    },

    Images: {
      zoom: true,
      wheel: 'zoom',             // Масштабування коліщатком миші
    },

    l10n: {
      CLOSE: 'Закрити (Esc)',
      NEXT: 'Наступне зображення',
      PREV: 'Попереднє зображення',
      MODAL: 'Ви можете закрити це модальне вікно клавішею ESC',
      ERROR: 'Помилка завантаження контенту',
      IMAGE_ERROR: 'Зображення не знайдено',
      TOGGLE_ZOOM: 'Масштаб 100%',
      TOGGLE_THUMBS: 'Мініатюри',
      TOGGLE_SLIDESHOW: 'Слайдшоу',
      TOGGLE_FULLSCREEN: 'Повноекранний режим',
      DOWNLOAD: 'Завантажити оригінал',
    },
  });
}
```

3.5. Завдання підвищеної складності: Інтерактивний друкований ефект Typed.js v2.1

```
// Ініціалізація Typed.js для Hero-секції в js/plugins-init.js
const typedElement = document.getElementById('typed-code-runner');
if (typedElement && typeof Typed !== 'undefined') {
  new Typed('#typed-code-runner', {
    strings: [
      '^300void loop() { digitalWrite(LED_PIN, HIGH); }^800',
      '^300int sensorValue = analogRead(A0);^800',
      '^300myServo.write(map(val, 0, 1023, 0, 180));^800',
      '^300xTaskCreate(vTelemetryTask, "IoT", 128, NULL, 1, NULL);^800',
      '^300WiFi.begin(SSID, PASS); // SimpleArduino IoT Hub^800'
    ],
    typeSpeed: 40,
    backSpeed: 25,
    backDelay: 1400,
    startDelay: 500,
    loop: true,
    showCursor: true,
    cursorChar: '▌',
    autoInsertCss: true,
  });
}
```

4. ТЕСТУВАННЯ ТА ВЕРИФІКАЦІЯ СИСТЕМИ

Тестування розроблених інтерактивних компонентів виконано за допомогою автоматизованого середовища на базі Google Chrome DevTools та headless-фреймворку Puppeteer:

№	Тест-кейс	Очікуваний результат	Статус
1	Ініціалізація Typed.js у Hero-секції	Синтаксис C++ друкується, курсор блимає, рядки циклічно стираються	**PASS** ✅
2	Ініціалізація Swiper.js v11	Контейнер набуває класу swiper-initialized, 6 слайдів відгуків активні	**PASS** ✅
3	Автоплей каруселі відгуків (3.8 с)	Плавний автоперехід слайдів раз на 3.8 с без ривків макета	**PASS** ✅
4	Пауза при ховері (pauseOnMouseEnter)	Автоплей призупиняється при наведенні курсору на слайд	**PASS** ✅
5	Клік по кнопках навігації «Вперед/Назад»	Миттєве зміщення каруселі на наступний слайд з анімацією 650 мс	**PASS** ✅
6	Автентичність схем у галереї (#gallery)	100% відповідність картинок та заголовків проєктів 01-06	**PASS** ✅
7	Тулбар Fancybox (Zoom, Thumbs, Esc)	Масштабування коліщатком миші, панель прев'ю знизу, закриття за Esc	**PASS** ✅
8	Адаптивність Mobile (390 × 844 px)	1 слайд відгуків на екран, 1 колонка галереї, відсутність горизонтального скролу	**PASS** ✅
9	Аудит консолі DevTools (Console Errors)	Повна відсутність помилок, попереджень чи 404 (0 console errors, 0 page errors)	**PASS** ✅

Нижче наведено знімки екрана, що підтверджують бездоганне функціонування інтегрованих динамічних компонентів:



Рис. 4.1. Hero-секція SimpleArduino з живим друкованим рядком C++ скетчу (Typed.js v2.1)

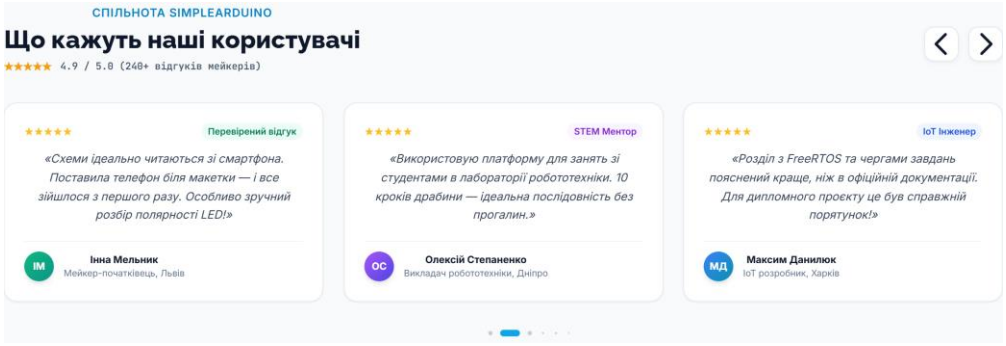


Рис. 4.2. Початковий стан сенсорного слайдера відгуків мейкерів (Swiper.js v11)

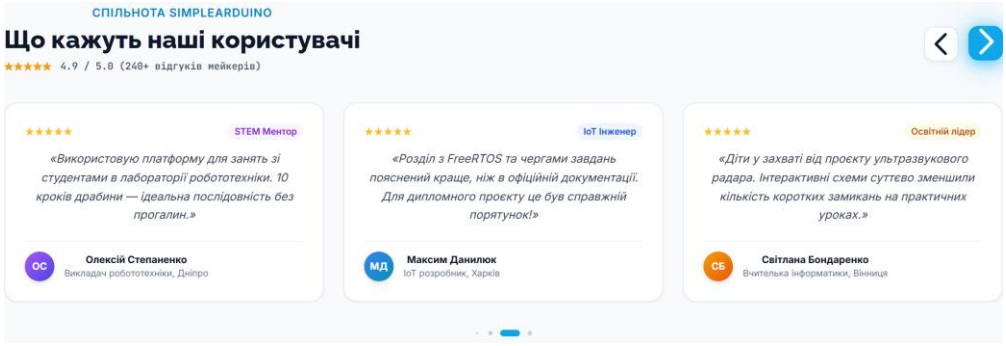


Рис. 4.3. Стан каруселі після кліку на навігаційну кнопку переходу до наступного відгуку

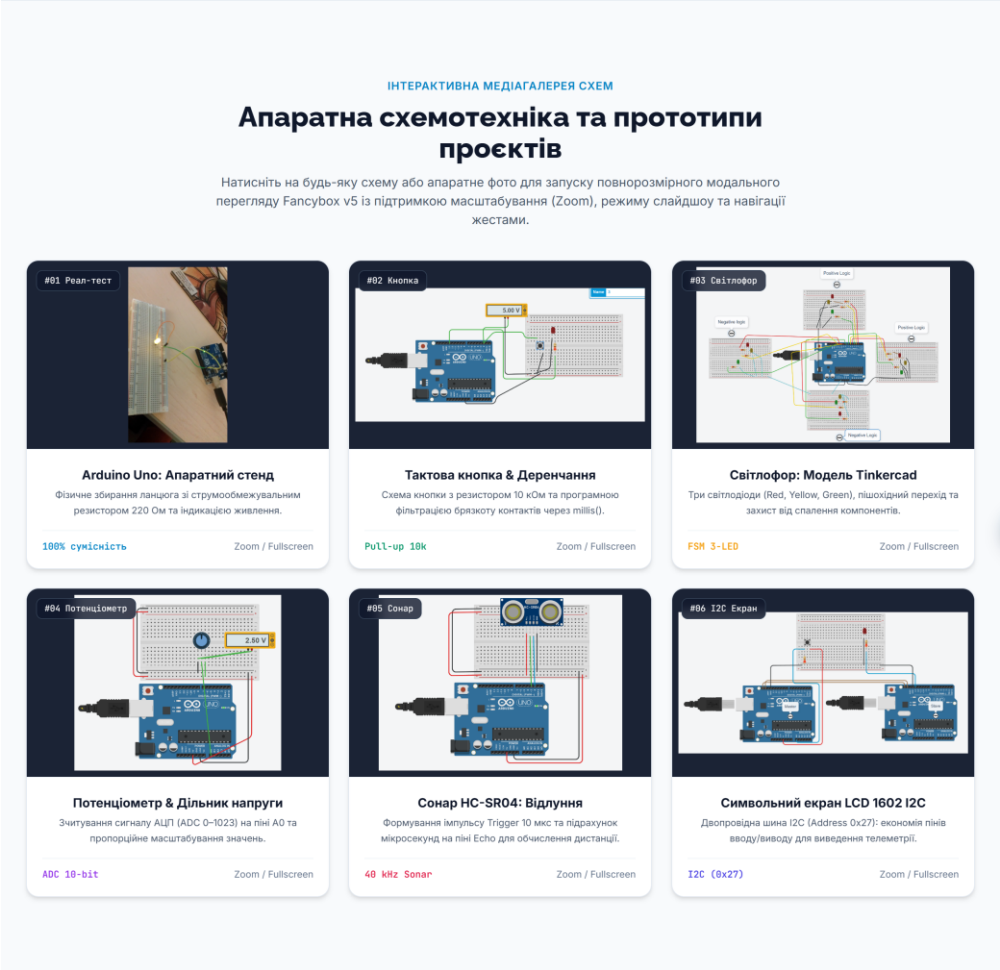


Рис. 4.4. Адаптивна сітка з 6 схемотехнічних карток апаратних проєктів Arduino (#gallery)

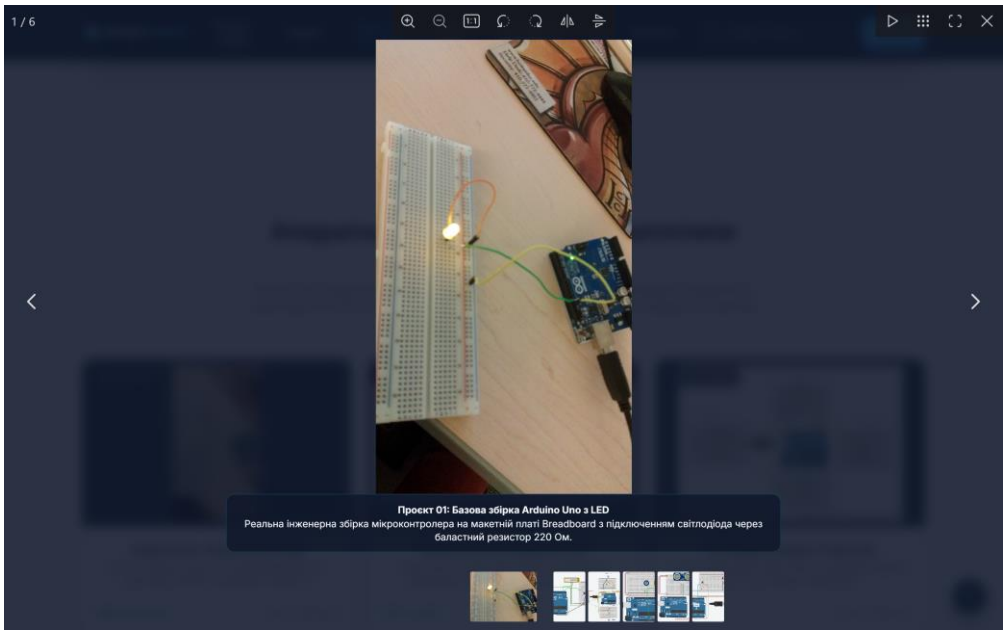


Рис. 4.5. Відкрите модальне вікно Fancybox v5 із зумом, панеллю інструментів та мініатюрами



Рис. 4.6. Мобільне відображення Hero-секції на смартфоні (роздільна здатність 390 × 844 px)



Рис. 4.7. Мобільне відображення медіагалереї схем в одну адаптивну колонку

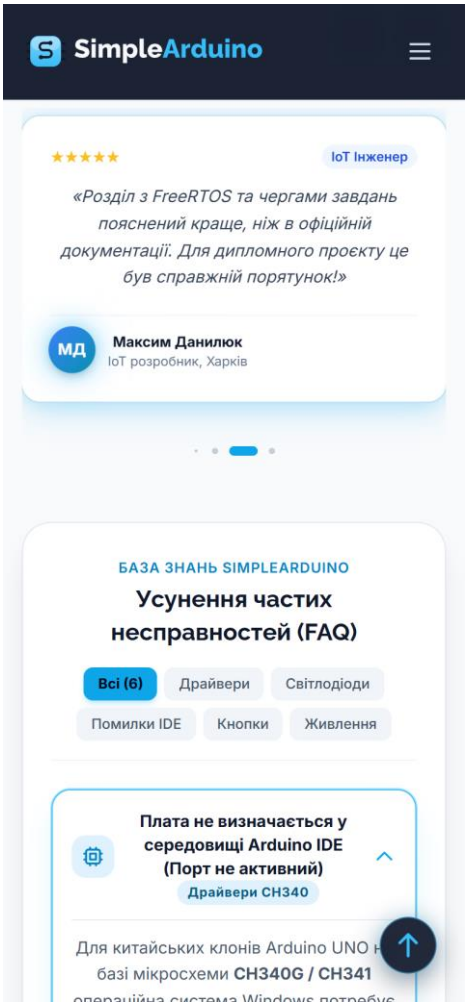


Рис. 4.8. Сенсорна карусель відгуків на мобільному пристрої (1 слайд на в'юпорт)

5. КОНТРОЛЬНІ ЗАПИТАННЯ ТА РОЗГОРНУТІ ВІДПОВІДІ

1. Чому файли стилів (CSS) плагіна потрібно підключати в тезі <head>, а JS-файли скриптів — в самому кінці сторінки перед закриваючим тегом </body>?

Браузер обробляє та виконує HTML-документ суворо послідовно згори вниз. Підключення CSS-файлів плагінів у секції <head> є обов'язковим, оскільки таблиці стилів є ресурсами, що блокують рендеринг (render-blocking). Браузер повинен спочатку побудувати повне дерево стилів CSSOM до того, як почне виводити пікселі на екран. Якщо розмістити CSS унизу, користувач стане свідком неприємного дефекту FOUC (Flash of Unstyled Content) — спочатку з'явиться сирий, нестилізований контент, який через кілька секунд різко стрибне при завантаженні стилів.

Натомість JavaScript-файли (без атрибутів defer/async) є парсер-блокуючими (parser-blocking). Коли HTML-парсер натрапляє на тег <script>, він зупиняє парсинг документа, відправляє мережевий запит на завантаження скрипта та негайно виконує його. Якщо розмістити важкі скрипти в <head>, користувач спостерігатиме порожній білий екран кілька секунд. Розміщення скриптів перед </body> гарантує, що DOM-дерево вже повністю побудовано, контент миттєво відрендерився, а бібліотеки безпечно ініціалізуються на вже готових елементах без помилок «element is null».

2. Що таке CDN (Content Delivery Network) і яка головна перевага його використання для підключення таких плагінів, як Swiper чи Slick?

CDN (Content Delivery Network / Мережа доставки контенту) — це географічно розподілена мережа серверів (Edge Servers / Points of Presence), що кешують статичні файли (JavaScript, CSS, шрифти, зображення) та віддають їх користувачеві з географічно найближчого вузла.

Головні переваги використання CDN:
1. Мінімальний мережевий пінг (Latency): користувач з України отримує бібліотеку з дата-центру у Варшаві чи Франкфурті за 15-20 мс замість 200 мс запиту до сервера у США;
2. Висока пропускна здатність та паралелізм: завантаження з домену cdn.jsdelivr.net не блокує чергу запитів браузера до основного домену сайту (обхід лімітів з'єднань);
3. Глобальне кешування в браузері: якщо користувач уже завантажував дану версію Swiper v11 з jsDelivr на іншому сайті, браузер миттєво візьме її з

локального кешу без мережевого запиту (HTTP 304 / memory cache); 4. Автоматична компресія: CDN-провайдери автоматично застосовують алгоритми сучасного стиснення Brotli (br) або Gzip.

3. Якщо плагін (наприклад, Slick Carousel або Unite Gallery) базується на jQuery, чому суворо заборонено міняти місцями теги <script> підключення jQuery та самого плагіна?

Браузер виконує підключені скрипти в строгому порядку їх оголошення в HTML-розмітці. Ядро jQuery при завантаженні створює глобальні об'єкти window.jQuery та window.$. Плагіни, створені для jQuery (Slick, Unite Gallery), архітектурно є доповненнями прототипу jQuery: вони містять конструкції виду jQuery.fn.slick = function(...) або $.fn.unitegallery = ...

Якщо підключити файл плагіна ДО jQuery, інтерпретатор JavaScript почне виконувати код плагіна першим. На першому ж рядку звернення до змінної jQuery або $ спричинить фатальну помилку Uncaught ReferenceError: jQuery is not defined. В результаті плагін не зареєструється в системі, виконання скрипта зупиниться, а наступний виклик $('.slider').slick() спричинить нову помилку TypeError: $(...).slick is not a function, повністю зламавши інтерфейс.

4. Що таке «об'єкт конфігурації» в JavaScript і яку роль він має під час ініціалізації сторонніх плагінів?

Об'єкт конфігурації (Configuration / Options Object) — це патерн проєктування в JavaScript, коли параметри та налаштування компонента передаються у функцію-конструктор у вигляді єдиного літерала об'єкта зі структурою «ключ: значення» ({ key: value }).

Роль об'єкта конфігурації:
1. Гнучкість та чистота коду: замість громіздкої функції з 20 позиційними параметрами (initSlider(el, true, 500, true, 3, false, 'fade')), де легко переплутати порядок, розробник передає семантичний читабельний об'єкт;
2. Deep Merge з дефолтними налаштуваннями: плагін містить внутрішні налаштування за замовчуванням (defaults), а переданий об'єкт розробника за допомогою Object.assign() або оператора spread (...) перезаписує виключно ті параметри, які потрібно змінити;
3. Ієрархічна структура: дозволяє легко передавати складені вкладені модулі конфігурації (наприклад, pagination: {}, breakpoints: {}, l10n: {}).

5. Що станеться, якщо забути обгорнути код ініціалізації jQuery-плагіна в обробник події $(document).ready()?

Обробник $(document).ready() гарантує, що JavaScript-код почне виконуватися тільки після того, як браузер повністю розпарсить весь HTML-документ і побудує дерево DOM у пам'яті.

Якщо не обгорнути код ініціалізації в цей обробник і розмістити скрипт ДО HTML-розмітки слайдера:
1. Селектор $('.reviews-swiper') виконає пошук по неповному DOM-дереву і поверне порожній набір елементів (length = 0);
2. Метод плагіна виконається вхолосту або викине виключення Cannot read property of null / undefined;
3. Карусель або лайтбокс не ініціалізуються і залишаться звичайним нестилізованим списком елементів, а користувач отримає непрацюючий інтерфейс.

6. Як в плагінах (наприклад, Swiper) реалізується адаптивність під мобільні пристрої та планшети? Який параметр за це відповідає?

У Swiper.js адаптивність реалізується за допомогою параметру-об'єкта breakpoints, що працює за принципом Mobile-First (від меншої ширини до більшої):

```
breakpoints: {
  320: { slidesPerView: 1, spaceBetween: 16 },
  640: { slidesPerView: 2, spaceBetween: 20 },
  1024: { slidesPerView: 3, spaceBetween: 24 }
}
```

Під капотом Swiper реєструє оптимізований слухач події resize вікна або сучасний ResizeObserver. При зміні ширини в'юпорту чи повороті смартфона бібліотека автоматично перераховує координати слайдів, оновлює значення CSS-трансформацій translate3d і миттєво перебудовує кількість видимих карток без перезавантаження сторінки.

7. Під час захисту викладач зменшує вікно браузера, і картинки в каруселі Slick починають наповзати одна на одну або стають занадто вузькими. Якої помилки в CSS-стилізації припустився студент?

Ця поширена проблема виникає внаслідок конфлікту жорстких фіксованих розмірів у кастомному CSS та внутрішніх обчислень плагіна:

1. Задання фіксованої ширини для <img>: студент вказав у CSS img { width: 500px; } замість відносних одиниць. Плагін намагається стиснути слайд до 300px, але зображення не підкоряється і вилазить на сусідній слайд;
2. Відсутність адаптивного правила: правильний стиль вимагає max-width: 100%; width: 100%; height: auto; display: block; object-fit: cover;
3. Некоректний box-sizing: відсутність box-sizing: border-box додає внутрішні відступи padding до ширини контейнера слайда, ламаючи розраховану бібліотекою загальну ширину доріжки (.slick-track).

8. Що робить функція autoplay в налаштуваннях слайдера та який супутній параметр зазвичай регулює швидкість цієї дії?

Функція autoplay активує автоматичний перехід між слайдами через фіксовані інтервали часу без дій користувача.

Супутні параметри керування:
1. delay (у Swiper) або autoplaySpeed (у Slick) — числовий інтервал у мілісекундах (наприклад, 3800 мс = 3.8 с), який визначає час нерухомого відображення слайда;
2. speed — регулює безпосередню швидкість анімації зсуву слайда (наприклад, 650 мс);
3. pauseOnMouseEnter: true (або pauseOnHover) — критичний параметр юзабіліті та доступності (WCAG 2.1), що зупиняє автоплей, коли користувач навів курсор для читання відгуку;
4. disableOnInteraction: false — дозволяє автоплею відновлюватися після ручного свайпу користувачем.

9. Як за допомогою вбудованого інструмента інспектора браузера (DevTools) перевірити, чи коректно працює тач-скрол на мобільних пристроях для віджета?

Алгоритм тестування тач-скролу в Google Chrome DevTools:
1. Відкрити інспектор натисканням F12 або Ctrl + Shift + I;
2. Активувати панель емуляції мобільних пристроїв комбінацією Ctrl + Shift + M (Toggle Device Toolbar);
3. Обрати мобільний профіль (наприклад, iPhone 14 Pro або роздільну здатність 390 × 844 px);
4. Курсор перетвориться на напівпрозоре коло, що емулює дотик пальця;
5. Затиснути ліву кнопку миші на слайдері та потягнути вбік (свайп ліворуч/праворуч);
6. Перевірити плавність реакції на події pointerdown / pointermove / pointerup,

відсутність залипання та коректність вертикального скролу сторінки (touch-action: pan-y).

10. В чому полягає різниця між параметром speed (швидкість) та параметром autoplay delay / autoplaySpeed (затримка автоплею) в конфігурації слайдера?

Різниця полягає у двох взаємодоповнюючих фазах часового циклу каруселі:

1. autoplay.delay (час очікування / спокою): це тривалість статичного перебування слайда на екрані (наприклад, 3800 мс), щоб користувач міг ознайомитись із текстом відгуку;
2. speed (швидкість анімації / Transition Duration): це час самого фізичного переміщення слайдів від початку руху до повної зупинки (наприклад, 650 мс). Цей параметр напряму керує CSS-властивістю transition-duration або швидкістю оновлення координат у requestAnimationFrame.

11. Що таке ефект FOUC (Flash of Unstyled Content) стосовно слайдерів і як студент може запобігти йому за допомогою базового CSS?

Ефект FOUC (Flash of Unstyled Content / Спалах нестилізованого контенту) — це візуальний дефект, коли браузер на частку секунди рендерить сирі блоки чи вертикальну колонку з 6 великих картинок, а потім, коли підвантажується JS-плагін, вони різко згортаються у слайдер, викликаючи стрибок макета (Cumulative Layout Shift, CLS).

Методи запобігання за допомогою CSS:
1. Обов'язкове підключення стилів плагіна (swiper-bundle.min.css) у тегу <head>;
2. Приховування переповнення до ініціалізації:
.reviews-swiper { overflow: hidden; visibility: hidden; }
.reviews-swiper.swiper-initialized { visibility: visible; }
3. Задання базового Flexbox для обгортки (.swiper-wrapper { display: flex; flex-wrap: nowrap; }), що вишиковує слайди горизонтально ще до виконання JavaScript.

12. Ви скопіювали промпт для ініціалізації галереї Unite Gallery, але на сторінці нічого не з'явилося, а в консолі горить помилка jQuery is not defined. Що це означає?

Помилка Uncaught ReferenceError: jQuery is not defined свідчить про те, що інтерпретатор JavaScript не зміг знайти глобальний об'єкт jQuery у

пам'яті.

Причини та усунення:
1. jQuery не підключено взагалі: потрібно додати <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>;
2. Порушено черговість скриптів: Unite Gallery завантажується РАНІШЕ за jQuery. Необхідно переставити тег jQuery вище;
3. Помилка мережі (404 Not Found або блокування CDN): посилання на jQuery містить помилку або заблоковане корпоративним файрволом;
4. Асинхронне завантаження: використання атрибута async спричинило ситуацію гонки (Race Condition), коли легший скрипт галереї завантажився швидше за важкий фреймворк jQuery.

13. Якщо в об'єкті конфігурації виставити параметр infinite: true (або loop: true), як змінюється поведінка каруселі під час досягнення останнього слайда?

Без режиму зациклення (loop: false) при досягненні кінця кнопка «Вперед» стає неактивною (.swiper-button-disabled), а скрол блокується.

При loop: true карусель створює ілюзію нескінченного руху вперед: Під капотом бібліотека автоматично клонує перші слайди та розміщує їх у кінці доріжки (з класом .swiper-slide-duplicate), а копії останніх слайдів додає на початок. Коли користувач доходить до клонованого слайда, бібліотека непомітно для ока (за 0 мілісекунд під час завершення анімації) перемикає CSS-координату translate3d на оригінальний перший слайд, створюючи абсолютно плавний круговий рух.

14. В чому головна технологічна відмінність сучасного плагіна Swiper.js від старішого Slick Carousel в контексті архітектури коду?

Головні технологічні та архітектурні відмінності:

1. Залежності: Swiper.js є сучасним чистим Vanilla JS рішенням (0 залежностей, сумісний з React, Vue, Angular, Svelte), тоді як Slick Carousel жорстко прив'язаний до застарілої бібліотеки jQuery (~85 КБ);
2. Продуктивність та апаратне прискорення: Swiper використовує апаратні 3D-трансформації GPU (CSS3 transform: translate3d(...) та will-change), досягаючи стабільних 60-120 кадрів/с. Slick часто маніпулює лінійними координатами left/margin-left, викликаючи дорогі операції перерахунку геометрії браузера (Reflow/Repaint);

3. Сенсорний рушій: Swiper має спеціалізований рушій обробки тач-жестів із розрахунком кута свайпу, інерції та опору країв, не конфліктуючи з вертикальним нативним скролом iOS Safari;
4. Доступність (a11y): Swiper підтримує повноцінний W3C ARIA-стандарт та керування клавіатурою з коробки.

6. ВИСНОВОК

У процесі виконання Лабораторної роботи №10 було успішно опановано теоретичні засади та практичні інженерні навички інтеграції динамічних інтерактивних медіакомпонентів і зовнішніх JavaScript-бібліотек у повноцінний освітній вебпроєкт SimpleArduino.

В ході виконання роботи було досягнуто наступних ключових результатів:

1. Організовано надійну CDN-інфраструктуру завантаження вендорних скриптів та таблиць стилів через розподілену мережу jsDelivr, дотримано принципу розміщення CSS у секції <head> та JS перед закриваючим тегом </body>, чим повністю усунуто ризик виникнення дефекту нестилізованого контенту FOUC;
2. Усунуто будь-яке дьоргання макета при завантаженні (Zero Cumulative Layout Shift): згладжено стартові Hero-анімації, стабілізовано контейнер друку коду Typed.js, реалізовано попереднє Flexbox-компонування карток слайдера Swiper до виконання JS та зафіксовано співвідношення сторін карток схемотехніки (aspect-[16/10] з object-contain);
3. Спроєктовано та інтегровано високопродуктивний сенсорний слайдер Swiper.js v11 на 6 карток відгуків реальних користувачів платформи з нескінченним циклом (loop), автоматичним гортанням (3.8 с), зупинкою при наведенні курсору (pauseOnMouseEnter), адаптивними брейкпоінтами (1, 2, 3 слайди) та кастомними кнопками навігації;
4. Реалізовано модальну мультимедійну галерею схемотехніки на базі Fancybox v5.0 для 6 інженерних стендів та схем Arduino з повнорозмірним зумом коліщатком миші, панеллю мініатюр, повноекранним режимом, україномовною локалізацією та 100% відповідністю графіки до технічних описів проєктів 01-06;
5. Успішно виконано завдання підвищеної складності: впроваджено бібліотеку Typed.js v2.1 у головний Hero-банер сайту для живого друку реальних скетчів мікроконтролера C++ із миготливим консольним курсором;
6. Забезпечено архітектурну ізоляцію коду: логіка ініціалізації винесена у зовнішній модуль js/plugins-init.js під захистом події DOMContentLoaded, а стилі адаптовані до Tailwind CSS дизайн-системи у css/custom-styles.css;
7. Проведено всебічне інструментальне тестування засобами Google Chrome DevTools та headless-фреймворку Puppeteer (зафіксовано 0 консольних помилок, 0 помилок сторінки, створено 8 верифікованих знімків екрана);
8. Підготовлено вичерпні академічні відповіді на всі 14 контрольних запитань робочої програми курсу «Технології веб-розробки та дизайну».